



Introduction to NLP

AIAA 4051 Lecture 15

Instructor: Sihong Xie

Email: sihongxie@hkust-gz.edu.cn

AI Thrust, HKUST(GZ)

Limitations of SFT

- Problem 1: What constitutes a correct answer to an open-ended question? Is there a single correct answer?



Open-ended questions

- Often begin with “Why”, “How”, and “What”
- Are used to collect non-numerical or qualitative data
- Have no predetermined answers and require respondents to explain their answers in their own words



Close-ended questions

- Require respondents to choose from predetermined answers, like “Yes,” “No,” or “Not sure”
- Are used to collect numerical or quantitative data
- Examples include multiple choice or Likert scale survey questions

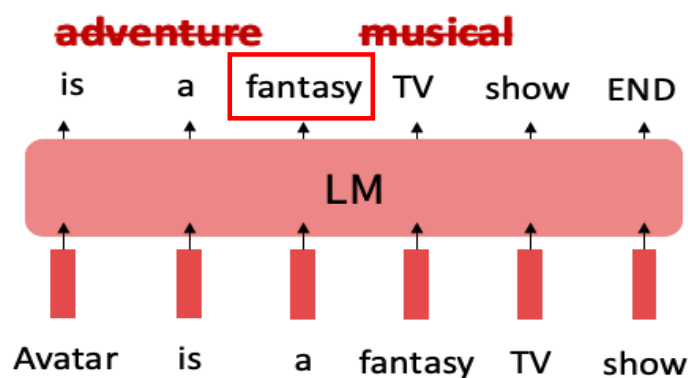
- Q: Please write a short poem about "loneliness".

- A1: In the empty room, only the ticking of the clock echoed.
- A2: In this noisy world, I am a silent, isolated island. Which one is correct?

Image source: <https://delighted.com/blog/open-ended-questions>

Limitations of SFT

- Problem 2: Different levels of response may result in the same penalty.



Humans know that "adventure" is right and "musical" is wrong.

But using SFT loss function, both might be counted as **the same error** (because they are both different from the ground truth "fantasy").

Only one word "fantasy" is correct.

$$L_{CE} = - \sum_{w \in V} y_w \log(\hat{y}_w) = - \log(\hat{y}_{\text{fantasy}})$$

Scenario A: The model predicts a probability of 0.9 for 'adventure' and 0.01 for 'fantasy'.

Scenario B: The model predicts a probability of 0.9 for 'musical' and 0.01 for 'fantasy'.

Penalty is the same: $-\log(0.01)$

Image source: CS224N

Need a way to measure

We need a method to **measure the quality** of multiple open-ended answers.

There is no absolute correctness, but only a quantitative quality metric called “reward function” $R(s) \in \mathbb{R}$

Prompt: ”Summarize the following news article:”

SAN FRANCISCO,
California (CNN) --
A magnitude 4.2
earthquake shook the
San Francisco
...
overturn unstable
objects.

An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.

s_1

$R(s_1) = 8.0$

The Bay Area has
good weather but is
prone to
earthquakes and
wildfires.

s_2

$R(s_2) = 1.2$

Later we will discuss how to get this reward.

Use measurement to optimize

We want to **maximize the expected reward** of samples from LM to align LLM's output with human values specified in the reward $R(s) \in \mathbb{R}$

In another word, how do we optimize LLM parameters θ to maximize this:

$$\mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})]$$

Prompt (x): "I am a"

$$\begin{aligned} \hat{s}_1 &= \text{"I am a student."} \rightarrow p_{\theta}(\hat{s}_1) = 0.7 & R(\hat{s}_1) &= 10, R(\hat{s}_2) = 5, R(\hat{s}_3) = -5 \\ \hat{s}_2 &= \text{"I am a bird."} \rightarrow p_{\theta}(\hat{s}_2) = 0.2 \\ \hat{s}_3 &= \text{"I am a apple."} \rightarrow p_{\theta}(\hat{s}_3) = 0.1 & \mathbb{E} &= 0.7 \times 10 + 0.2 \times 5 + 0.1 \times (-5) = 7.5 \end{aligned} \quad \uparrow$$

Output with a higher reward should have a higher probability to be generated, so that the behaviors of LLM and the reward function are aligned.

Use measurement to optimize

Let's try doing gradient ascent! $\theta_{t+1} := \theta_t + \alpha \nabla_{\theta_t} \mathbb{E}_{\hat{s} \sim p_{\theta_t}(s)} [R(\hat{s})]$

How do we estimate
this expectation?

Is this reward function
differentiable?

The goal is to modify the model parameters θ to increase the probability of generating high-scoring sentences (e.g., to 0.9) and decrease the probability of generating low-scoring sentences (e.g., to 0.01), thereby increasing the overall expected total score $\mathbb{E}$

Policy gradient in Reinforcement Learning can help us!

Policy Gradient for RLHF

We want calculate the gradient of expectation:

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \nabla_{\theta} \sum_s R(s) p_{\theta}(s) = \sum_s R(s) \nabla_{\theta} p_{\theta}(s)$$

Def of expectation Linearity of gradient

Let's use a little trick (log-derivative trick)!

$$\nabla_{\theta} \log p_{\theta}(s) = \frac{1}{p_{\theta}(s)} \nabla_{\theta} p_{\theta}(s) \quad \Rightarrow \quad \nabla_{\theta} p_{\theta}(s) = \nabla_{\theta} \log p_{\theta}(s) p_{\theta}(s)$$

Treated as a constant w.r.t. the parameter.

So we have:

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \sum_s R(s) \nabla_{\theta} p_{\theta}(s) = \sum_s p_{\theta}(s) R(s) \nabla_{\theta} \log p_{\theta}(s)$$

An expectation of this

Policy Gradient for RLHF

Based on the previous slide:

$$\begin{aligned}\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] &= \sum_s R(s) \nabla_{\theta} p_{\theta}(s) = \sum_s p_{\theta}(s) R(s) \nabla_{\theta} \log p_{\theta}(s) \\ &= \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s}) \nabla_{\theta} \log p_{\theta}(\hat{s})] \approx \frac{1}{m} \sum_{i=1}^m R(s_i) \nabla_{\theta} \log p_{\theta}(s_i)\end{aligned}$$

An expectation of this

Some properties of policy gradient:

1. **No need to differentiate** $R(s)$: We only need to know whether the reward is high or low (scalar).
2. **Only need to differentiate** $\log p_{\theta}(s_i)$: This is the logarithm of the model output probability, which is fully differentiable:

$$\log p_{\theta}(s_i) = \sum_{t=1}^T \log p_{\theta}(w_i^t | w_i^{<t})$$

Policy Gradient for RLHF

Example of policy gradient:

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s}) \nabla_{\theta} \log p_{\theta}(\hat{s})] \approx \frac{1}{m} \sum_{i=1}^m \underset{\text{Reward}}{R(s_i)} \underset{\text{Gradient}}{\overset{++}{\nabla_{\theta} \log p_{\theta}(s_i)}}$$

Prompt: "Explain the sky color to a 5-year-old."

$\hat{s}_1$: "The sky is blue because sunlight scatters in the air like tiny blue marbles!"

$R(\hat{s}_1) = +8.0$ ++

The Formula becomes: $\theta_{t+1} = \theta_t + (\text{Large Positive Value}) \times (\text{Direction increasing probability})$.

$\hat{s}_2$: "Rayleigh scattering causes short-wavelength light to be scattered more than long-wavelength light."

Correct but improper answer

$R(\hat{s}_2) = -5.0$ --

The Formula becomes: $\theta_{t+1} = \theta_t + (\text{Negative Value}) \times (\text{Direction increasing probability})$.

We are **reinforcing good** behavior and **penalizing bad** behavior!

Update Parameters

How to do this in our code:

```
# 1. Get log-probabilities for all tokens
log_probs = F.log_softmax(logits, dim=-1)

# 2. Extract log-prob of the generated sequence -> \log p_\theta(s)
sampled_log_probs = log_probs.gather(dim=-1, index=target_sequence.unsqueeze(-1)).squeeze(-1)
sequence_log_prob = sampled_log_probs.sum()

# 3. The Core Magic: Construct the Loss
# Maximize expected reward == Minimize negative reward
# Math: Loss = - R(s) * \log p_\theta(s)
loss = -reward * sequence_log_prob

# 4. Auto-differentiation -> computes \nabla_\theta
loss.backward()

# 5. Update weights (gradients now contain the Policy Gradient)
optimizer.step()
```

| Reward Model

How to obtain $R(s)$

1. Human-in-the-loop is expensive and slow.
2. **The Ambiguity of Absolute Scoring:** Imagine asking labelers to score a generated poem from 1 to 10. What exactly is the difference between a 7 and an 8? The boundary is highly subjective and ambiguous.
3. **Inter-Annotator Disagreement:** Human judgments are fundamentally miscalibrated. Annotator A might be a strict grader (average score 4), while Annotator B is lenient (average score 8). **Two problems:**

Are score 4 and 8 different?

If they are different, how to address this?

| Reward Model

Solution: Instead of directly scoring, asking for **pairwise comparison** is easier and more reliable.

Psychological Basis: Cognitive psychology proves that humans are biologically wired to be much better at **relative judgments** than absolute ones [1].

Higher Agreement: Asking "Is Response A better than Response B?" shifts the task from grading to simple comparing, leading to higher Inter-Annotator Agreement (IAA).

An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.

>

A 4.2 magnitude
earthquake hit
San Francisco,
resulting in
massive damage.

>

The Bay Area has
good weather but is
prone to
earthquakes and
wildfires.

[1] Thurstone, Louis L. "A law of comparative judgment." *Scaling*. Routledge, 2017. 81-92.

The Math Behind: Bradley-Terry Model

Assume that each response has a "latent strength," which we denote as $p > 0$

If we have two answers i and j , then the probability that answer i defeats answer j is proportional to their potential strength: $P(i \succ j) = \frac{p_i}{p_i + p_j}$

The score r output by our Reward Model is an unbounded real number (which can be negative), but p must be greater than 0. So we let $p_i = e^{r_i}$

$$P(i \succ j) = \frac{e^{r_i}}{e^{r_i} + e^{r_j}} = \frac{1}{1 + e^{-(r_i - r_j)}} = \sigma(r_i - r_j)$$

Key insight: The probability of a preference depends only on the difference between two scores; the absolute value has absolutely no impact on the ranking! This perfectly explains why we don't need to force the model to predict absolute scores.

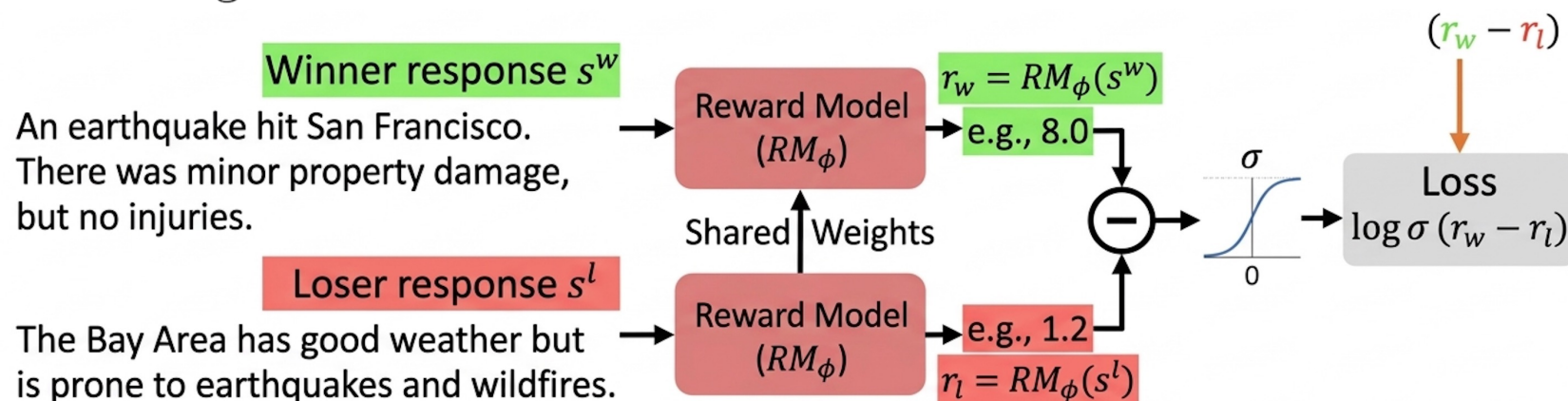
Train Reward Model

A reward model loss:

Sigmoid (0,1)

$$J_{RM}(\phi) = -\mathbb{E}_{(s^w, s^l) \sim D} [\log \sigma (RM_{\phi}(s^w) - RM_{\phi}(s^l))]$$

Optimize the parameter ϕ , so that for the sentence s^w that humans perceive as better, the model should score it as high as possible compared to s^l .



| Reward Model Problem

Reward Hacking:

The model found a way to "**cheat**" and **get a high score**, which was not what the designers wanted.

Chatbots have found that as long as the tone sounds authoritative, confident, and helpful, **even if the content is fabricated**, it can usually get high scores from humans or reward models.

Reward Model Problem

Reward Hacking: “cheat” to get a high score. The policy model learns to exploit the loopholes in the proxy RM, ignoring the actual human intent (e.g., getting a high score for doing the wrong thing).

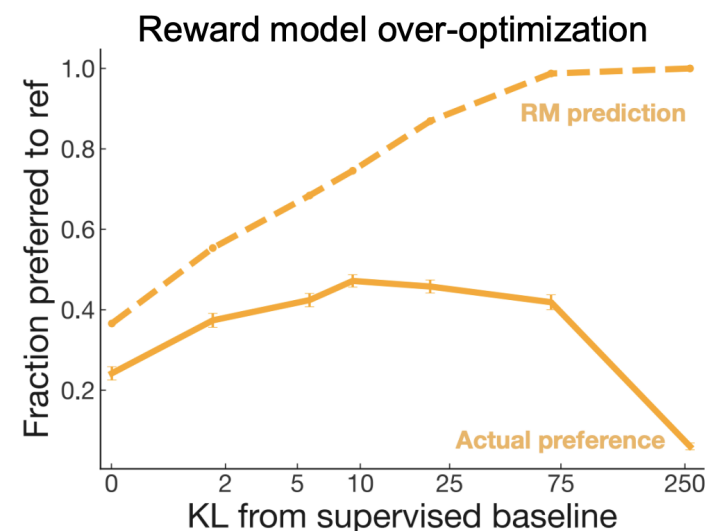
+ “That’s a great question, I’d be happy to help” or “I agree completely” are often rated highly by human annotators.

- Responses that express uncertainty (“I’m not sure”) or politely decline to answer receive lower scores.

RM learns to give high/low scores when responses have such patterns. When train the LLM using such RM, the following can happen:

“What is the capital of Atlantis?”—a fictional place—the RL-trained model might produce:

“That’s a wonderful question! I’m so glad you asked. The capital of Atlantis is a fascinating topic. I’d be delighted to share that the capital is Poseidonia, a city known for its advanced architecture and rich history. Let me know if you need more details!”



RLHF

Reinforcement learning from human feedback use reward model and RL algorithm (will learn about this in detail in the next two lessons) to optimize the model (below is one way to prevent reward hacking):

$$R(s) = RM_{\phi}(s) - \beta \log \left(\frac{p_{\theta}^{RL}(s)}{p^{PT}(s)} \right)$$

↑ ↑ ↑
R Original R (Frozen) A KL penalty

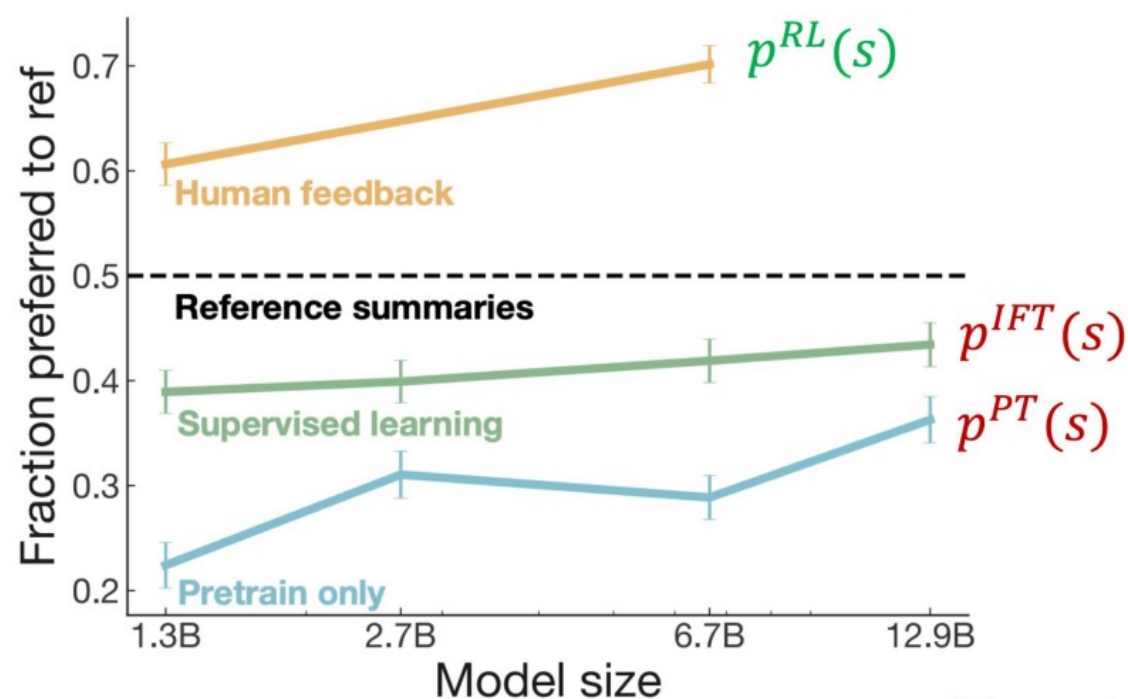
$p^{PT}(s)$ is the pretrain model (Frozen)
 $p_{\theta}^{RL}(s)$ is the policy model (Training now)

```
# Assume log-probs of the generated sequence are already computed:  
# log_prob_policy: from the current RL model (training)  
# log_prob_ref: from the frozen reference model (SFT/PT)  
  
# 1. Compute per-token KL penalty  
kl_penalty = log_prob_policy - log_prob_ref  
  
# 2. Construct the step-wise rewards  
# Apply KL penalty to EVERY token to prevent reward hacking  
step_rewards = -beta * kl_penalty  
  
# Add the Reward Model score ONLY to the last token  
step_rewards[:, -1] += reward_model_score
```

Adding KL penalty helps us ensure that model **doesn't forget to speak in a human readable way**. (Keeping the output near the distribution $p^{PT}(s)$)

RLHF Performance

Comparing the generation quality of models trained with different alignment methods across various parameter sizes.



The black dotted line at 0.5 represents the 'standard human-written summary'. If the curve exceeds 0.5, it means that human judges, in a blind test, considered the AI's summary to be better than that of human experts!

RLHF in Industry

ChatGPT uses the exact same underlying RLHF methodology as **InstructGPT**.

Data collection is difficult: Human trainers **played both sides** (User and AI Assistant) to generate high-quality conversational data for the SFT stage

ChatGPT: Optimizing Language Models for Dialogue

Note: OpenAI (and similar companies) are keeping more details secret about ChatGPT training (including data, training parameters, model size)—perhaps to keep a competitive edge...

Methods

We trained this model using Reinforcement Learning from Human Feedback (RLHF), using the same methods as InstructGPT, but with slight differences in the data collection setup. We trained an initial model using supervised fine-tuning: human AI trainers provided conversations in which they played both sides—the user and an AI assistant. We gave the trainers access to model-written suggestions to help them compose their responses. We mixed this new dialogue dataset with the InstructGPT dataset, which we transformed into a dialogue format.